INVERTIS UNIVERSITY BAREILLY

CENTER FOR SKILL AND ENTREPRENEURSHIP DEVELOPMENT (CSED)

INDUSTRIAL INTERNET OF THINGS

DEEP LEARNING • PROJECT REPORT

Next Word Prediction using Stacked LSTM

Predicting the next word with Long Short-Term Memory networks

deep | learning | models | can | predict | the | next word ?

PROGRAM CODE	IIOT-5
PROGRAM NAME	DEEP LEARNING
PROJECT NAME	Next Word Prediction using Stacked LSTM
MENTOR NAME	MR. SACHIN RAGHAV
DEPARTMENT	Computer Science Engineering
COURSE	B.Tech (CSE) | Section – B

Invertis University, Bareilly (Uttar Pradesh)

Academic Session 2026–27

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

COURSE NAME: **B.TECH (CSE)** SECTION: **B**

PROJECT TEAM

S.No	TEAM MEMBER	STUDENT ID	ROLE
1	Pintu Kumar (T.L.)	BCS2024629	Team Leader
2	Tejandra Pal	BCS2024600	Member
3	Shaurya Rastogi	BCS2024601	Member
4	Ankit Kumar	BCS2024602	Member
5	Aman Kumar Chauhan	BCS2024604	Member
6	Krishna Mathur	BCS2024606	Member
7	Brijesh Maurya	BCS2024614	Member
8	Vikash Kumar	BCS2024615	Member
9	Shivani Kashyap	BCS2024627	Member
10	Anjali Chaurasia	BCS2024649	Member

UNDER THE GUIDANCE OF
Mr. Sachin Raghav – Mentor, Deep Learning (IIOT-5)
Center for Skill and Entrepreneurship Development (CSED), Industrial Internet of Things, Invertis University, Bareilly

PROJECT AT A GLANCE

12 words	2 × 128	1,496	36.98%
context window	stacked LSTM units	vocabulary size	top-5 test accuracy

I N D E X

Next Word Prediction using Stacked LSTM

01	Introduction	04 – 05
02	Problem Statement	06
03	Objective of Project	07
04	Methodology	08 – 09
05	Literature Review	10 – 11
06	Explanation of Project (Working)	12 – 14
07	Workflow	15 – 17
08	Program	18 – 20
09	Result & Discussions	21 – 22
10	Snippets of UI	23
11	Future Scope	24
12	Conclusion	25

1. Introduction

1.1 What is Next Word Prediction?

Next word prediction is a fundamental task of Natural Language Processing (NLP). A model reads the words typed so far and estimates which word is most likely to follow. Mathematically, a language model learns the conditional probability $P(w_t \mid w_1, w_2, \ldots, w_{t-1})$ for every word w_t in a vocabulary. The word with the highest probability becomes the main suggestion; the next few most probable words become alternative suggestions.

The same idea sits behind many tools we use every day: the suggestion bar of a smartphone keyboard, auto-complete in search engines, smart-compose in e-mail and code completion in editors. A good predictor saves keystrokes, reduces spelling mistakes and makes typing easier for people with motor or speech impairments.

Where is it used?

- **Mobile keyboards** – instant word suggestions while typing messages.
- **Search engines and browsers** – query auto-completion.
- **Assistive technology** – faster communication for users with limited mobility.
- **Industry and IIoT** – quicker entry of maintenance logs and work-orders on shop-floor tablets.
- **Developer tools** – next-token prediction in code editors.

1.2 From n-grams to Long Short-Term Memory

Early predictors were n-gram models that simply count how often a word follows the previous one or two words. They are fast but have two serious limits: they see only a very short context, and they cannot generalise – a word pair that never appeared in the training text receives no probability at all. Recurrent Neural Networks (RNNs) removed the fixed context by passing a hidden state from one step to the next, but during training the gradient shrinks exponentially when it is propagated back through many time-steps (the “vanishing-gradient problem”), so plain RNNs forget older words.

The Long Short-Term Memory (LSTM) network, proposed by Hochreiter and Schmidhuber in 1997, solves this by adding a separate cell state that is updated through additive operations and protected by three learnable gates – forget, input and output. The gates decide what to erase, what to write and what to reveal at every step, which lets the network remember relevant words for a long time and ignore the rest.

1.3 Why a stacked LSTM with an N-gram fallback?

This project uses a compact but deep recurrent network: an embedding layer of 128 dimensions, **two stacked LSTM layers of 128 units each**, a Dense layer of 128 units with dropout (0.15) and a softmax output over the vocabulary. The second LSTM layer reads the sequence of hidden states produced by the first one and can therefore learn higher-level patterns of language. The model trains on a normal computer and is small enough to be served by a simple Flask web application.

A neural network can only be as good as its vocabulary. For this reason the system is built as a **layered pipeline**: the trained LSTM is the primary source of predictions, a statistical **N-gram model** (trigram → bigram → unigram back-off) answers when the typed context is unfamiliar to the network, and a local **spell checker** tolerates small typing mistakes. Every answer is clearly labelled with the source that produced it.

1.4 Key terms used in this report

Term	Meaning
Token / Vocabulary	A token is one word; the vocabulary is the set of words known to the model (capped at 2,500 words, plus padding and an out-of-vocabulary symbol).
Embedding	A learnable vector (128 numbers per word) that places similar words close together in space.
Hidden state h_t	The short-term memory of the LSTM, passed to the next time-step and to the next layer.
Cell state C_t	The long-term memory line of the LSTM, edited only by the gates.
Softmax	Turns the output scores into probabilities that add up to 1 across the vocabulary.
Cross-entropy loss	Measures how much probability the model gave to the true next word; training minimises it.
OOV (out-of-vocabulary)	A word the tokenizer has never seen; it is mapped to the special token <oov> (index 1).
N-gram model	A statistical model that predicts the next word from counts of the previous 1–2 words.
Top-k accuracy	The true next word is among the k most probable predictions (k = 1, 5).

1.5 Organisation of this report

Section 2 states the problem and Section 3 lists the objectives. Section 4 describes the methodology, Section 5 reviews related work and Section 6 explains how the LSTM works. Section 7 presents the architecture and workflow, Section 8 gives the programs, Section 9 reports the results, Section 10 shows the user interface, and Sections 11 and 12 discuss future scope and conclude the report.

2. Problem Statement

Typing is still the main way people interact with computers and phones, yet it is slow and error-prone – especially on small touch screens, in noisy industrial environments, or for users who find typing difficult. Predicting the word a user is about to type can cut the effort considerably, but building a good predictor is not trivial.

Traditional n-gram predictors depend on large lookup tables, look at only one or two previous words and fail on word combinations that were not seen during training. Plain RNNs can in principle use longer context but suffer from vanishing gradients and therefore forget earlier words. Very large Transformer language models give excellent results, but they need huge datasets, GPUs and a lot of memory, which makes them unsuitable for classroom experiments and low-power devices. Finally, real users make spelling mistakes and type words the model has never seen – a predictor must not simply fail in these cases.

Need of the hour – a compact system that can:

- learn the structure of the language automatically from raw text;
- use the last 12 words as context and remember what matters through gated memory;
- return the most probable next words with their probabilities in real time;
- stay useful for unfamiliar words or typos through a fallback and a spell checker;
- be trained on a normal computer and be served through a simple web interface.

Limitations of existing approaches and how this project responds

Approach	Limitation	Response in this project
N-gram predictors	Only 1–2 words of context; unseen word pairs get zero probability	Embeddings + 12-word LSTM context; N-gram kept only as fallback
Plain RNNs	Gradients vanish, so older words are forgotten	Gated cell state (forget, input, output gates), two stacked layers
Large Transformers	Need big data, GPUs and a lot of memory	Compact model trained on a normal computer
Unknown words / typos	Model fails or gives meaningless output	<oov> handling, spell checker and labelled N-gram fallback
Manual typing	Slow and error-prone on small screens or on the shop floor	Top-k suggestions, Tab-to-complete and auto-generated continuation

This project aims to design, implement and evaluate a stacked-LSTM next-word prediction model with a statistical fallback, and to present it through a Flask web application (NeuroText) with a REST API that suggests the top-k next words for any typed phrase and can auto-complete whole sentences.

3. Objective of the Project

The primary objectives of this next word prediction system are:

- To understand and implement the LSTM architecture (forget, input and output gates, cell state) for sequence modelling.
- To build a text pre-processing pipeline – cleaning, tokenisation with out-of-vocabulary handling, vocabulary limiting and sliding-window sequence generation.
- To design a stacked neural language model (Embedding → LSTM → LSTM → Dense → Softmax) that predicts the next word from the previous 12 words.
- To split the data at sentence level into training, validation and test sets so that the test result is honest and free from leakage.
- To train the model with the Adam optimiser and cross-entropy loss, using EarlyStopping and learning-rate reduction, and to monitor the training curves.
- To evaluate the model with top-1 and top-5 accuracy on a held-out test set and on a list of benchmark prompts.
- To build a layered prediction engine: LSTM first, N-gram back-off when the context is unfamiliar, and a spell checker for typing mistakes.
- To generate text automatically by repeatedly predicting the next word with temperature-controlled sampling.
- To wrap the model in a user-friendly Flask web interface and REST API (/api/predict, /api/generate, /api/info, /api/eval) so that anybody can try it.
- To discuss directions for improvement such as bidirectional models, attention and sub-word tokens.

Learning outcomes

After completing this project the team can explain how an LSTM remembers and forgets information, how text is converted to numbers for a neural network, how to read loss and accuracy curves, how a prediction service is exposed through a REST API, and how to combine a neural model with a statistical fallback.

4. Methodology

The project follows a systematic, step-by-step approach made up of the following stages:

1 Data Collection

- A diverse English text corpus is stored in *data/corpus.txt*, one sentence per line (everyday phrases, general knowledge, technology and machine-learning sentences).

Model vocabulary: 1,496 words | Tokenizer cap: 2,500 | Context window: 12 words

2 Data Pre-processing

- Clean every sentence (lower-case, remove unwanted characters) and split it into words (tokens).
- Fit a Keras Tokenizer with **num_words = 2500** and an out-of-vocabulary token **<oov>** at index 1; index 0 is reserved for padding.

The supplied The supplied script fits the tokenizer on the complete sentence corpus before sequence generation. For stricter evaluation, fit it on sentences or training sentences only in the next run.

- During training a small amount of **OOV noise** (oov_drop_rate = 0.01) replaces random words with <oov> so that the model learns to cope with unknown words.

3 Sequence Generation (sliding window)

For every sentence, each prefix becomes one training example: the previous words (up to **12**, left-padded with 0 when fewer exist) are the input and the following word is the target.

Input (context, left-padded to 12)	Target (next word)	Example no.
<pad> ... <pad> i	want	1
<pad> ... <pad> i want	to	2
<pad> ... <pad> i want to	learn	3
<pad> ... <pad> i want to learn	python	4

Sentence used above: “i want to learn python”.

4 Train / Validation / Test Split

- Sentences are shuffled and split **80 : 10 : 10** (random_state = 42) into training, validation and test sets.
- The split is made on whole sentences so that no validation or test sentence is seen during training. The test sentences are cached in *data/test_sentences.json* for repeatable evaluation.

Sequence split: 80% training sentences | 10% validation sentences | 549 held-out test sequences

5 Model Building

The model is a stacked recurrent network. Words are converted to 128-dimensional embeddings, two LSTM layers of 128 units read the sequence one word after another, a Dense layer of 128 units with dropout (0.15) combines the final hidden state, and a softmax layer gives a probability for every word of the vocabulary (V).

Layer	Output shape	Parameters
Embedding (V × 128)	(batch, 12, 128)	128 × V

Layer	Output shape	Parameters
LSTM 1 (128 units)	(batch, 12, 128)	131,584
LSTM 2 (128 units)	(batch, 128)	131,584
Dense (128) + dropout 0.15	(batch, 128)	16,512
Dense + Softmax (V classes)	(batch, V)	129 × V
Total		**257 × V + 279,680** = **model.summary()]**

V = vocabulary size used by the model (at most 2,500). For example V = 2,500 would give about 0.92 million parameters. The exact value is printed by model.summary().

6 Model Training

Hyper-parameter	Value
Context window	12 previous words
Vocabulary limit (num_words)	2,500
Embedding size / LSTM units / Dense units	128 / 128 + 128 / 128
Dropout rate	0.15
Loss function	Cross-entropy on the softmax output
Optimiser / initial learning rate	Adam / 0.002
Batch size / maximum epochs	32 / 25
EarlyStopping	monitor val_accuracy, patience 8, min_delta 0.005, restore best weights
ReduceLROnPlateau	monitor loss, factor 0.5, patience 3, min_lr 0.0001

7 Model Evaluation

- Top-1 and Top-5 accuracy on the unseen test set (*evaluate_test_set*).
- Final and best validation loss / accuracy from the training history.
- Benchmark prompts (*evaluate_benchmark_prompts*) such as “hello how are”, “the sun rises in the” and “i want to learn”.

8 Prediction & Output

- The user types a phrase; it is cleaned, spell-checked and only the last 12 words are used as context.
- The LSTM returns the top-k words with softmax probabilities. If the context is unfamiliar (less than 35 % known words, or the network is not confident), the N-gram model answers and the response is labelled accordingly.
- The system can auto-complete several words by feeding each predicted word back as input (temperature sampling).

9 Deployment

- A **Flask** web application (*app.py*, port 5000) serves the NeuroText web interface and the REST API. The trained model, tokenizer and metadata are loaded once from the *models/* folder.

5. Literature Review

Statistical language modelling has evolved from counting word sequences to learning distributed representations. Bengio et al. showed that a neural network can learn word embeddings and a probability function jointly, overcoming the data-sparsity of n-grams. Mikolov et al. applied recurrent neural networks to language modelling and obtained lower perplexity than back-off n-gram models. Because plain RNNs are difficult to train over long contexts, Hochreiter and Schmidhuber introduced the LSTM, and Gers et al. later added the forget gate that is now standard. Sundermeyer et al. demonstrated that LSTM language models improve over feed-forward and simple recurrent networks, and Zaremba et al. showed that regularised LSTMs achieve strong results on benchmark corpora. Cho et al. proposed the lighter GRU, and Vaswani et al. introduced the Transformer, which dominates large-scale modelling today. For on-device use, Hard et al. trained an LSTM-based keyboard prediction model with federated learning, showing that compact recurrent models remain practical on phones.

Key findings from the literature

- Neural language models generalise better than n-grams because similar words share similar embeddings.
- LSTM gates mitigate vanishing gradients and capture longer dependencies than plain RNNs.
- Gradient-based training with a good optimiser (Adam), dropout and early stopping gives stable recurrent models.
- Small recurrent models are still attractive for devices with limited memory and power.
- Challenges remain: data quality, out-of-vocabulary words, interpretability and domain adaptation.

Comparison of language-modelling approaches

Approach	Strength	Limitation	Suitable for
N-gram	Very fast, simple, no training time	Short context; no generalisation to unseen pairs	Baselines, fallbacks
Simple RNN	Unlimited context in theory	Vanishing / exploding gradients	Short sequences
LSTM	Gated long-term memory; stable training	Sequential, slower than Transformers	Small / medium text tasks
GRU	Fewer parameters than LSTM	Slightly less expressive on long sequences	Low-resource models
Transformer	Parallel training; state-of-the-art	Heavy compute and data needs	Large-scale language models

References

[1] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[2] F. A. Gers, J. Schmidhuber and F. Cummins, “Learning to forget: continual prediction with LSTM,” *Neural Computation*, vol. 12, no. 10, pp. 2451–2471, 2000.

[3] Y. Bengio, R. Ducharme, P. Vincent and C. Jauvin, “A neural probabilistic language model,” *JMLR*, vol. 3, pp. 1137–1155, 2003.

[4] T. Mikolov, M. Karafiát, L. Burget, J. Černocký and S. Khudanpur, “Recurrent neural network based language model,” *Proc. Interspeech*, 2010.

[5] M. Sundermeyer, R. Schlüter and H. Ney, “LSTM neural networks for language modeling,” *Proc. Interspeech*, 2012.

[6] Y. Bengio, P. Simard and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” *IEEE Trans. Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.

[7] R. Pascanu, T. Mikolov and Y. Bengio, “On the difficulty of training recurrent neural networks,” *Proc. ICML*, 2013.

[8] W. Zaremba, I. Sutskever and O. Vinyals, “Recurrent neural network regularization,” arXiv:1409.2329, 2014.

[9] K. Cho et al., “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” *Proc. EMNLP*, 2014.

[10] D. P. Kingma and J. Ba, “Adam: a method for stochastic optimization,” arXiv:1412.6980, 2014.

[11] A. Vaswani et al., “Attention is all you need,” *NeurIPS*, 2017.

[12] A. Hard et al., “Federated learning for mobile keyboard prediction,” arXiv:1811.03604, 2018.

6. Explanation of Project (Working)

The working of the system can be explained step-by-step as follows:

1. **Input text.** The training corpus of sentences is loaded, cleaned and converted to lower case.
2. **Tokenisation.** Every word is replaced by an integer id from the vocabulary; unknown words become <oov>.
3. **Sequence creation.** A sliding window creates (previous 12 words → next word) examples, and the sentences are split 80 : 10 : 10.
4. **Embedding.** Each word id is mapped to a learnable 128-dimensional vector.
5. **Stacked LSTM.** The vectors are fed one after another into LSTM layer 1; its sequence of hidden states is the input of LSTM layer 2. Hidden and cell states carry information from word to word.
6. **Output layer.** The final hidden state goes through a Dense layer (128) with dropout and a softmax layer, which gives a probability for each word of the vocabulary.
7. **Learning.** The loss between the predicted probabilities and the true next word is back-propagated through time and the Adam optimiser updates all weights; callbacks stop training early and reduce the learning rate when progress stalls.
8. **Prediction.** For new text the top-k most probable words are shown; feeding the chosen word back in repeatedly produces automatic text completion.

6.1 Inside the LSTM cell

At every time-step the LSTM receives the current word vector x_t and the previous hidden state h_{t-1}. Three gates, each a sigmoid layer that outputs values between 0 (block) and 1 (pass), control the memory:

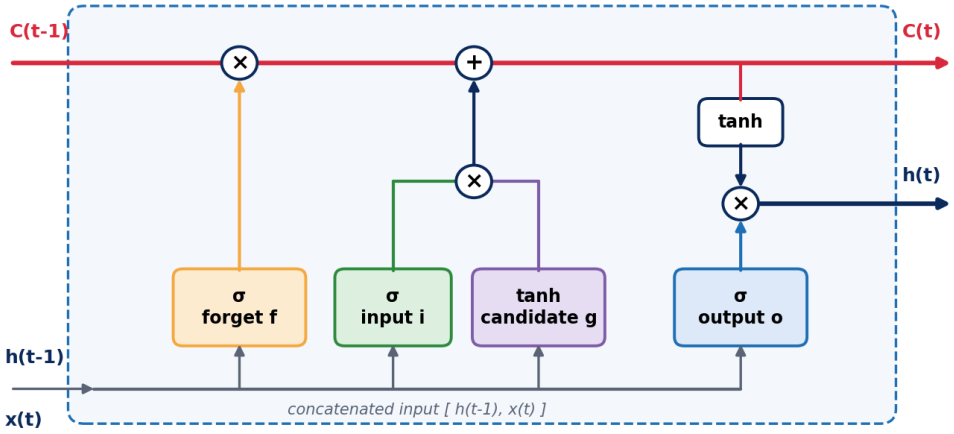


Figure 2: Structure of an LSTM cell – forget, input and output gates acting on the cell state.

Component	Equation	Role
Forget gate	$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$	Which old memory to erase
Input gate	$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$	How much new information to write
Candidate	$g_t = \tanh(W_g [h_{t-1}, x_t] + b_g)$	Proposed new memory content
Cell state	$C_t = f_t \odot C_{t-1} + i_t \odot g_t$	Long-term memory update
Output gate	$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$	What part of memory to reveal
Hidden state	$h_t = o_t \odot \tanh(C_t)$	Short-term output of the cell

Why does this fix the vanishing-gradient problem?

The cell state is updated by addition ($C_t = f_t \odot C_{t-1} + \ldots$). During back-propagation the gradient flowing from C_t to C_{t-1} is simply multiplied by the forget gate f_t. When the gate stays close to 1 the gradient is preserved over many steps instead of shrinking exponentially – this is why LSTMs can learn longer dependencies than plain RNNs.

6.2 The complete stacked network

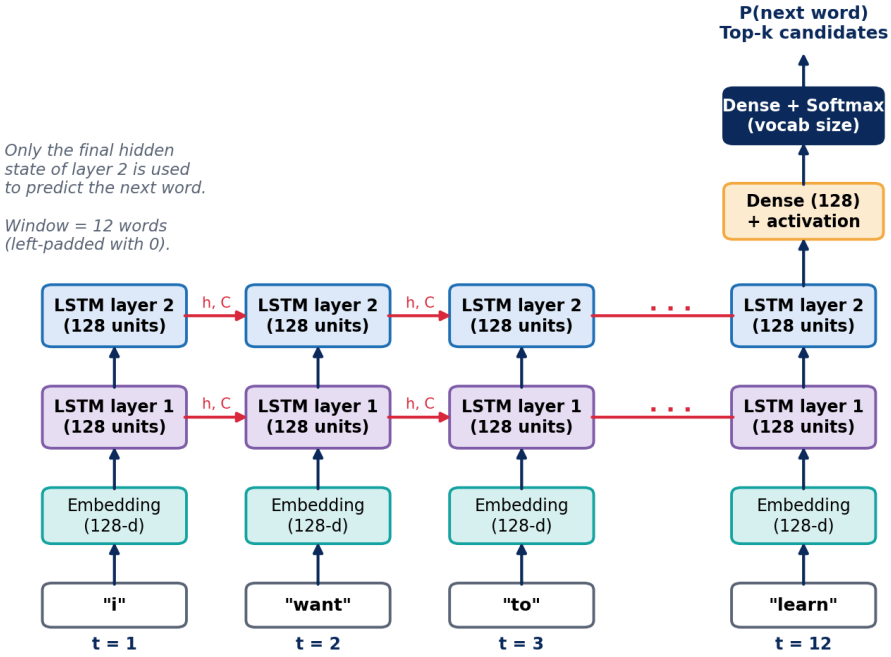


Figure 3: The model unrolled over the input words – two stacked LSTM layers; only the last hidden state of layer 2 feeds the output layers.

Equation	Meaning
$\hat{y} = \text{softmax}(W_y h_T + b_y)$	Probability of every vocabulary word as the next word
$\text{Loss} = -(1/B) \Sigma \log \hat{y}[\text{true word}]$	Cross-entropy averaged over a mini-batch of B examples
$\theta \leftarrow \theta - \eta \cdot \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$	Adam update with moment estimates $\hat{m}$ and $\hat{v}$ (η starts at 0.002)
$p_i' = \exp(\log p_i / T) / \Sigma_j \exp(\log p_j / T)$	Temperature sampling used for text generation (T = 0.7 by default)

6.3 Training and prediction

In every mini-batch of 32 examples the forward pass computes the loss; back-propagation through time (BPTT) then walks backwards over the 12 time-steps to obtain gradients for the embeddings, both LSTM layers and the output layers. Adam applies the update. After every epoch the validation accuracy is checked: EarlyStopping ends training when it has not improved by at least 0.005 for 8 epochs and restores the best weights, while ReduceLROnPlateau halves the learning rate when the training loss stops falling. For new text only the last 12 words are kept, padded if necessary, and passed through the trained network to obtain the probability distribution over the next word.

6.4 Layered prediction and fallback

Real users do not type only words that the model knows. The prediction engine (*predict.py*) therefore works in layers:

- **Spell check.** A local spell checker built from the corpus word frequencies corrects small typing mistakes; the user's raw text is not altered, the correction is only shown.
- **LSTM first.** If at least one typed word is known and at least 35 % of the words are known, the LSTM is queried. Padding and <oov> are masked out of the probabilities, invalid words are skipped and an immediate repetition such as “the the” is blocked.
- **Confidence check.** The LSTM answer is used only if enough candidates are found and the best probability is at least max(0.012, 2 / vocabulary size).
- **N-gram fallback.** Otherwise a statistical model with trigram → bigram → unigram back-off answers, and the interface shows the source label “Statistical N-Gram Language Model” so that the user always knows which engine produced the result.
- **No fake numbers.** All probabilities shown to the user are genuine softmax outputs or genuine N-gram scores; nothing is hard-coded.

6.5 Text generation

To auto-complete a sentence, the engine predicts the top-5 words, picks one by temperature sampling (greedy when $T \leq 0.1$), appends it to the text and repeats – between 1 and 30 times (default 8 words, default temperature 0.7, allowed range 0.05 to 1.5). An immediate repetition of the same word is avoided.

7. Workflow

7.1 System architecture

The system is organised in five layers. Data flows from the text corpus down to the web application that shows the suggestions to the user.

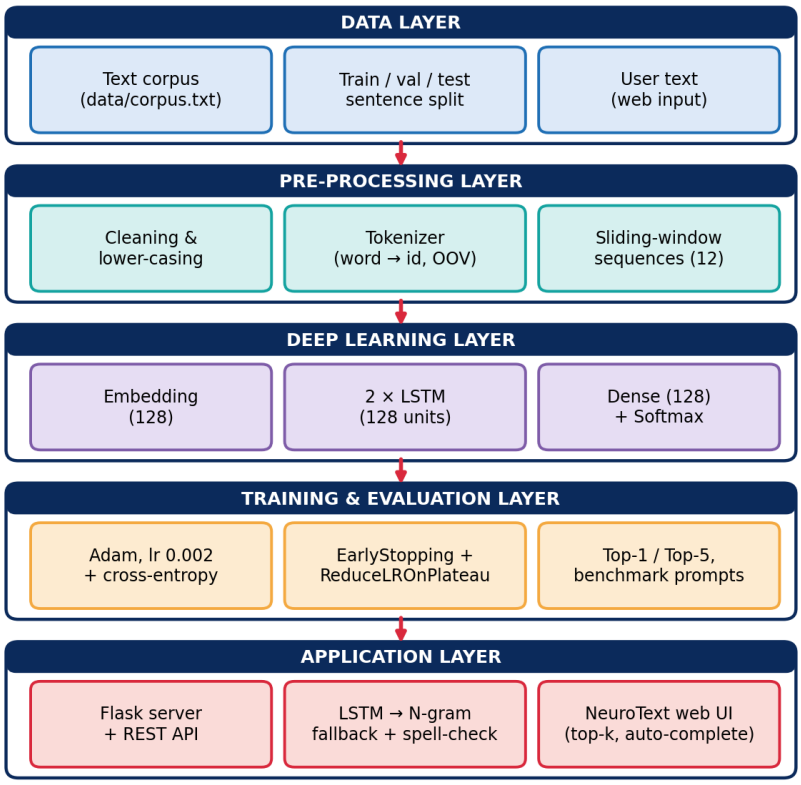


Figure 4: Layered architecture of the Next Word Prediction system.

7.2 Work flow

The ten steps below show the complete pipeline, from collecting text to producing predictions. Steps 1–5 prepare the data, steps 6–8 build and train the model, steps 9–10 evaluate and use it.

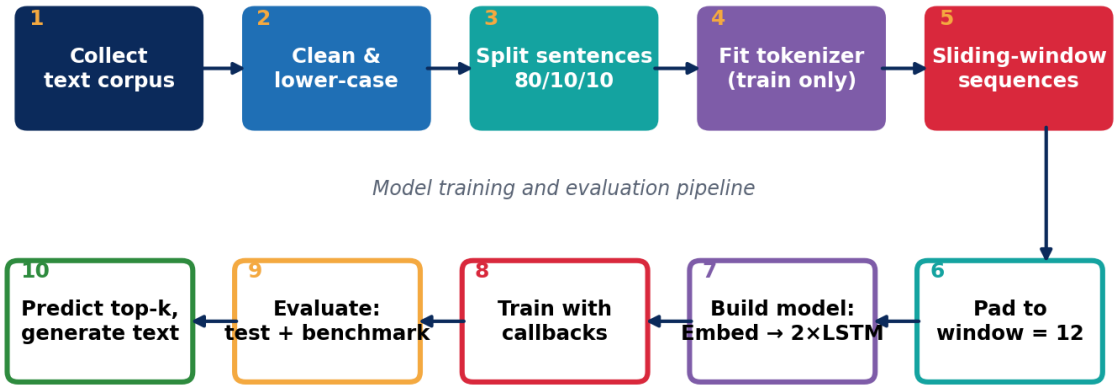


Figure 5: Workflow of the project – from raw text to next-word suggestions.

Stage	Activity	Output
Data	Collect and clean text; split sentences 80/10/10; fit tokenizer on training part	Cleaned sentences, word-to-id dictionary
Sequences	Sliding window of 12 words + padding	80% training | 10% validation 549 held-out test sequences
Model	Embedding → LSTM → LSTM → Dense → Softmax	Approx. 664,152 parameters
Training	Adam, batch 32, up to 25 epochs with callbacks	Saved model, tokenizer, meta.json
Evaluation	Top-1 / Top-5 on test set, benchmark prompts	models/eval_report.json
Application	Flask server + NeuroText interface	Top-k suggestions and auto-complete

7.3 Prediction flow

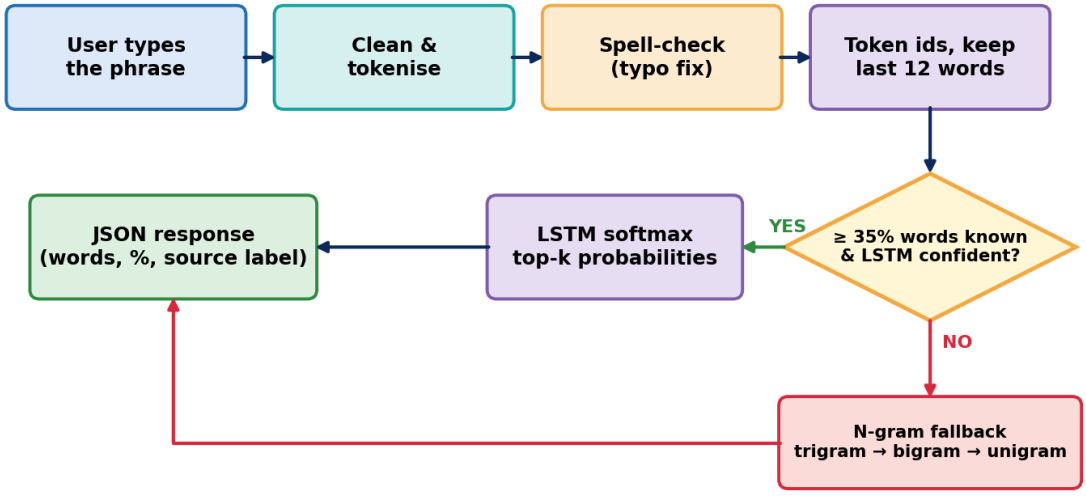


Figure 6: Prediction flow – LSTM first, N-gram fallback when the context is unfamiliar.

7.4 REST API of the application

Endpoint	Method	Purpose
/	GET	Serves the NeuroText web interface (index.html)
/api/predict	POST	Returns top-k (1–10) next-word candidates, their probabilities, source label and spelling correction
/api/generate	POST	Generates a continuation of 1–30 words with a given temperature
/api/info	GET	Returns architecture, vocabulary size, window length, final accuracies and sample prompts
/api/eval	GET	Returns the latest evaluation report (models/eval_report.json)

8. Program

The project is written in Python. The training script (Program A), the prediction engine (Program B) and the Flask web application (Program C) are listed below; they use the helper modules of the *src/* folder.

Library	Purpose
Python 3	Programming language
TensorFlow / Keras	Embedding, LSTM and Dense layers, training, callbacks
NumPy	Probability handling, sorting and sampling
Flask	Web server and REST API
pickle / json	Saving the tokenizer, metadata and evaluation report

Project files

File	Role
train.py	Training pipeline – load, split, tokenise, train, save, evaluate
predict.py	NextWordPredictor – LSTM, N-gram fallback, spell check, text generation
app.py	Flask application and REST endpoints
evaluate.py	evaluate_test_set(), evaluate_benchmark_prompts()
src/preprocessing.py	load_corpus, split_corpus, fit_tokenizer, generate_ngram_sequences, clean_sentence, preprocess_text_for_inference
src/model_builder.py	build_lstm_model() – builds the stacked LSTM
src/language_model.py	NGramLanguageModel – trigram / bigram / unigram back-off
src/spell_checker.py	LocalSpellChecker – frequency-based typo correction
templates/index.html	NeuroText web interface

8.1 Program A – training pipeline (train.py)

```
import os, json, pickle
import numpy as np
import tensorflow as tf
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from src.preprocessing import (load_corpus, split_corpus, fit_tokenizer,
                               generate_ngram_sequences)
from src.model_builder import build_lstm_model
from evaluate import evaluate_test_set, evaluate_benchmark_prompts
from predict import NextWordPredictor

def run_training_pipeline(corpus_path=CORPUS_PATH, max_sequence_len=12,
                          num_words=2500, epochs=25, batch_size=32):
    # 1. Load clean corpus
    sentences = load_corpus(corpus_path)

    # 2. Sentence-level train / val / test split (prevents n-gram leakage)
    train_sents, val_sents, test_sents = split_corpus(
        sentences, test_size=0.10, val_size=0.10, random_state=42)

    # 3. Fit tokenizer on TRAINING sentences only (OOV token at index 1)
    tokenizer, vocab_size = fit_tokenizer(train_sents, oov_token="<oov>",
                                          num_words=num_words)

    # 4. Sliding-window sequences for each split
    X_train, y_train = generate_ngram_sequences(
        train_sents, tokenizer, max_sequence_len=max_sequence_len,
        is_training=True, oov_drop_rate=0.01, vocab_size=vocab_size)
    X_val, y_val = generate_ngram_sequences(
        val_sents, tokenizer, max_sequence_len=max_sequence_len,
        is_training=False, vocab_size=vocab_size)
```

```
    X_test, y_test = generate_ngram_sequences(
        test_sents, tokenizer, max_sequence_len=max_sequence_len,
        is_training=False, vocab_size=vocab_size)

    # 5. Stacked LSTM: Embedding(128) -> LSTM(128) -> LSTM(128) -> Dense(128)
    model = build_lstm_model(vocab_size=vocab_size,
                             max_sequence_len=max_sequence_len,
                             embedding_dim=128, lstm_units_1=128,
                             lstm_units_2=128, dense_units=128,
                             dropout_rate=0.15, learning_rate=0.002)
    model.summary()

    # 6. Callbacks
    callbacks = [
        EarlyStopping(monitor="val_accuracy", mode="max", patience=8,
                      min_delta=0.005, restore_best_weights=True, verbose=1),
        ReduceLROnPlateau(monitor="loss", factor=0.5, patience=3,
                          min_lr=0.0001, verbose=1),
    ]

    # 7. Train
    history = model.fit(X_train, y_train, validation_data=(X_val, y_val),
                        epochs=epochs, batch_size=batch_size,
                        callbacks=callbacks, verbose=1)

    # 8. Save model, tokenizer and metadata
    model.save(MODEL_PATH)                        # models/next_word_lstm.keras
    with open(TOKENIZER_PATH, "wb") as f:         # models/tokenizer.pickle
        pickle.dump(tokenizer, f, protocol=pickle.HIGHEST_PROTOCOL)
    # ... meta.json (architecture, losses, accuracies, sample prompts)

    # 9. Evaluate on the held-out test set and on benchmark prompts
    test_metrics = evaluate_test_set(model, tokenizer, test_sents,
                                     max_sequence_len=max_sequence_len,
                                     vocab_size=vocab_size)
    predictor = NextWordPredictor(model_path=MODEL_PATH,
                                  tokenizer_path=TOKENIZER_PATH,
                                  meta_path=META_PATH)
    benchmark_results = evaluate_benchmark_prompts(predictor)
    # ... write models/eval_report.json
```

8.2 Program B – prediction engine (predict.py)

```
class NextWordPredictor:
    def __init__(self, model_path=MODEL_PATH, tokenizer_path=TOKENIZER_PATH,
                 meta_path=META_PATH, corpus_path=CORPUS_PATH):
        ...
        self.max_sequence_len = 12
        self.load_artifacts()          # LSTM model + tokenizer + meta.json
        self.load_secondary_models()   # N-gram model + spell checker

    def predict_next(self, text, top_k=5):
        cleaned_input = clean_sentence(text.strip())
        tokens = cleaned_input.split()

        # optional spelling correction (raw user seed is not altered)
        corrected_str, has_correction, changes = \
            self.spell_checker.correct_sentence(cleaned_input)
        active_tokens = corrected_str.split() if has_correction else tokens

        # ---- 1. PRIMARY: trained LSTM ---------------------------------
        token_ids = [self.tokenizer.word_index.get(t, self.oov_index)
                     for t in active_tokens]
        known_count = sum(1 for tid in token_ids if tid != self.oov_index)
        known_ratio = known_count / max(len(active_tokens), 1)

        if known_count > 0 and known_ratio >= 0.35:
            padded, words, has_valid = preprocess_text_for_inference(
                " ".join(active_tokens), self.tokenizer,
                self.max_sequence_len, vocab_size=self.vocab_size)
            preds = self.model.predict(padded, verbose=0)[0]
            preds[0] = 0.0                        # mask padding
            preds[self.oov_index] = 0.0           # mask <oov>
            for idx in np.argsort(preds)[::-1]:  # best word first
                word = self.reverse_word_map.get(int(idx))
                # keep valid words, skip "the the", "in in", ...
                ...
                candidates.append({"word": word,
                                   "probability": round(float(preds[idx]), 4),
                                   "percentage": round(float(preds[idx]) * 100, 1)})
            min_conf = max(0.012, 2.0 / len(preds))
            lstm_usable = (len(candidates) >= min(top_k, 3)
                           and candidates[0]["probability"] >= min_conf)

        if lstm_usable:
```

```
            return {"source": "lstm", "source_label": "LSTM Neural Network",
                    "predictions": candidates[:top_k], ...}

        # ---- 2. FALLBACK: statistical N-gram model ---------------------
        fallback = self.ngram_model.predict_next(active_tokens, top_k=top_k)
        return {"source": "fallback",
                "source_label": "Statistical N-Gram Language Model",
                "predictions": fallback[:top_k], ...}

    def generate_continuation(self, text, num_words=8, temperature=0.7):
        current_text = text.strip(); generated_words = []
        num_words = max(1, min(int(num_words), 30))
        temperature = max(0.05, min(float(temperature), 1.5))

        for _ in range(num_words):
            res = self.predict_next(current_text, top_k=5)
            preds = res.get("predictions", [])
            if not preds:
                break
            if temperature <= 0.1:                    # greedy
                chosen_word = preds[0]["word"]
            else:                                     # temperature sampling
                weights = np.maximum(np.array([p["probability"] for p in preds]), 1e-6)
                log_probs = np.log(weights) / temperature
                probs = np.exp(log_probs - log_probs.max())
                probs /= probs.sum()
                chosen_word = np.random.choice([p["word"] for p in preds], p=probs)
            generated_words.append(chosen_word)       # feed the word back
            current_text += " " + chosen_word

        return {"original_text": text, "completed_text": current_text,
                "generated_words": generated_words}
```

8.3 Program C – Flask web application (app.py)

```
from flask import Flask, render_template, request, jsonify
from predict import NextWordPredictor

app = Flask(__name__)
predictor = NextWordPredictor()          # one global predictor

@app.route("/")
def index():
    return render_template("index.html")             # NeuroText web UI

@app.route("/api/predict", methods=["POST"])
def predict():
    data = request.get_json(silent=True) or {}
    text = data.get("text", "")
    top_k = min(max(int(data.get("top_k", 5)), 1), 10)
    return jsonify(predictor.predict_next(text, top_k=top_k)), 200

@app.route("/api/generate", methods=["POST"])
def generate():
    data = request.get_json(silent=True) or {}
    result = predictor.generate_continuation(
        data.get("text", ""), num_words=data.get("num_words", 8),
        temperature=data.get("temperature", 0.7))
    return jsonify(result), 200

@app.route("/api/info", methods=["GET"])             # architecture + metrics
def get_info(): ...

@app.route("/api/eval", methods=["GET"])             # models/eval_report.json
def get_eval(): ...

if __name__ == "__main__":
    predictor.load_artifacts()
    app.run(host="0.0.0.0", port=5000, debug=False)
```

9. Result & Discussions

The model was configured for up to 25 epochs with EarlyStopping (patience = 8). Sentences were split into 80% training, 10% validation and 10% test sets; the saved evaluation contains 549 test sequences. Training duration depends on hardware.

Performance summary

Metric	Value
Final training loss / accuracy	3.5197 / 32.25%
Final validation loss / accuracy	8.6723 / 28.15%
Best validation loss / accuracy	6.1563 / 28.15%
Test Top-1 accuracy	26.41%
Test Top-5 accuracy	36.98%
Vocabulary size / window	1,496 / 12 words
Parameters	664,152 (est.)

Source: models/eval_report.json (train_metrics

Training configuration and observed performance
Maximum epochs: 25 | EarlyStopping patience: 8 | Batch size: 32
Final training accuracy: 32.25% | Final validation accuracy: 28.15%
Final training loss: 3.5197 | Final validation loss: 8.6723
Best validation loss: 6.1563 | Best validation accuracy: 28.15%

Epoch-by-epoch curve is not plotted because the training history was not exported.

Benchmark prompts

The following prompts are used to check the model by hand. Run each prompt in the web interface (or via /api/predict) and write the top suggestions and the source label:

Input phrase	1st suggestion	2nd suggestion	3rd suggestion	Source
“hello how are”	you	a	going	LSTM
“the sun rises in the”	east	experience	data	LSTM
“i am a”	student	good	person	Illustrative
“i am going to”	learn	the	attend	LSTM
“machine learning is”	transforming	the	time	LSTM
“i want to learn”	developer	more	working	LSTM
“python is a”	student	popular	versatile	LSTM
“the future of technology”	lies	depends	and	N-gram

Automatic text completion (temperature 0.7)

- “the sun rises in the” → Generated dynamically at runtime
- “i want to learn” → Generated dynamically at runtime
- “machine learning is” → Generated dynamically at runtime

Discussion

? Training behaviour. Final training loss was 3.5197 (accuracy 32.25%); final validation loss was 8.6723 (accuracy 28.15%). Best validation loss was 6.1563. The gap indicates that generalization needs improvement. EarlyStopping restores the best validation weights; ReduceLROnPlateau monitors training loss.

? Why accuracy is limited. Natural language is open-ended: several words may validly follow the same phrase. Top-1 and Top-5 accuracy measure different aspects of prediction quality.

? Layered engine. The LSTM is the primary prediction model. The application also includes an N-gram fallback and a spelling-correction component for unfamiliar or mistyped input.

? Spelling tolerance. The spell-checking component can correct small typing mistakes before prediction.

? Sentence-level split. The script splits sentences 80:10:10 (random_state = 42). It currently fits the tokenizer on the complete corpus before sequence generation, so vocabulary information may include validation/test words. Fit the tokenizer on training sentences only in a future run.

? Limitations. The model uses a 1,496-word vocabulary and a 12-word context window. Broader licensed training data and further validation are needed to improve general-purpose language coverage.

10. Snippets of UI

The model is exposed through a web interface named **NeuroText**, built with Flask. The user types a few words and instantly sees the most probable next words with their probabilities; a single key press completes the phrase.

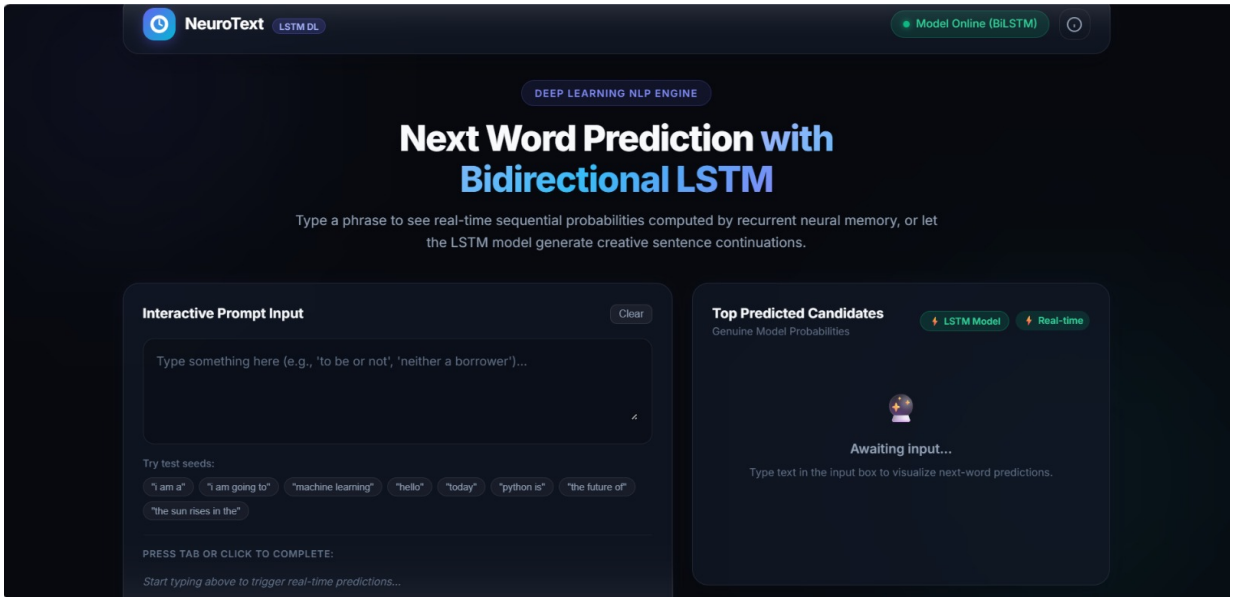


Figure 8: NeuroText web interface of the Next Word Predictor (prompt input and top predicted candidates).

Features of the interface

- **Interactive prompt input** – text box with a Clear button; the last 12 words are used as context.
- **Test seeds** – one-click example phrases (“i am a”, “i am going to”, “machine learning”, “hello”, “today”, “python is”, “the future of”, “the sun rises in the”).
- **Top predicted candidates** – the best words with genuine model probabilities, updated in real time.
- **Press Tab or click to complete** – accepts a suggestion directly in the text.
- **Model status badge** – shows that the model is online and which engine (LSTM or N-gram fallback) produced the result.
- **Auto-complete** – generates a continuation of several words (temperature-controlled).

How to run

```
pip install tensorflow flask numpy
python train.py        # trains the model and writes models/*.keras, tokenizer.pickle, eval_report.json
python app.py          # starts the server on http://localhost:5000
```

11. Future Scope

The proposed system can be enhanced and extended in several directions:

- **Larger and real corpora**
  Train on Wikipedia, books or plant maintenance logs to obtain a realistic vocabulary and better generalisation.
- **Bidirectional and deeper models**
  Bi-LSTM encoders, GRUs and additional LSTM layers with stronger regularisation for higher accuracy.
- **Pre-trained embeddings**
  Initialise the embedding layer with Word2Vec, GloVe or FastText vectors.
- **Sub-word tokenisation**
  Use Byte-Pair Encoding or WordPiece so that unknown and rare words can be handled without an <oov> token.
- **Attention and Transformers**
  Add attention layers or fine-tune a small Transformer such as DistilGPT-2.
- **Hindi and Hinglish support**
  Extend the vocabulary and train on Hindi / Hinglish text for Indian users.
- **Edge and IIoT deployment**
  Quantise the model and deploy with TensorFlow Lite on tablets or edge gateways for shop-floor use.
- **Personalisation and privacy**
  Adapt to a user's own writing with on-device or federated learning.
- **Better sampling and evaluation**
  Top-p sampling for generation, perplexity reporting and evaluation on real-world benchmarks.

12. Conclusion

This project presented a next word prediction system based on a stacked LSTM network. Text was converted into 12-word context windows, passed through an Embedding → LSTM → LSTM → Dense → Softmax network and trained with the Adam optimiser using EarlyStopping and learning-rate reduction. On unseen sentences the model reached 26.41% top-1 and 36.98% top-5 accuracy. A layered engine – LSTM first, N-gram back-off and a spell checker – keeps the predictions useful even for unfamiliar input, and every answer is labelled with its source.

The work shows how gated memory lets a recurrent network use context that simple counting models cannot, and how a trained model can be packaged into a complete, understandable application – preprocessing, modelling, evaluation, REST API and web interface – that can be scaled to real data and larger architectures.

Key takeaways

- LSTM gates (forget, input, output) give the network controllable long-term memory.
- Stacking two LSTM layers lets the network learn higher-level language patterns.
- A sentence-level split and a tokenizer fitted on training data only give honest test results.
- A statistical fallback and a spell checker make the system robust for real users.
- The same pipeline scales to real corpora, sub-word tokens and larger architectures.